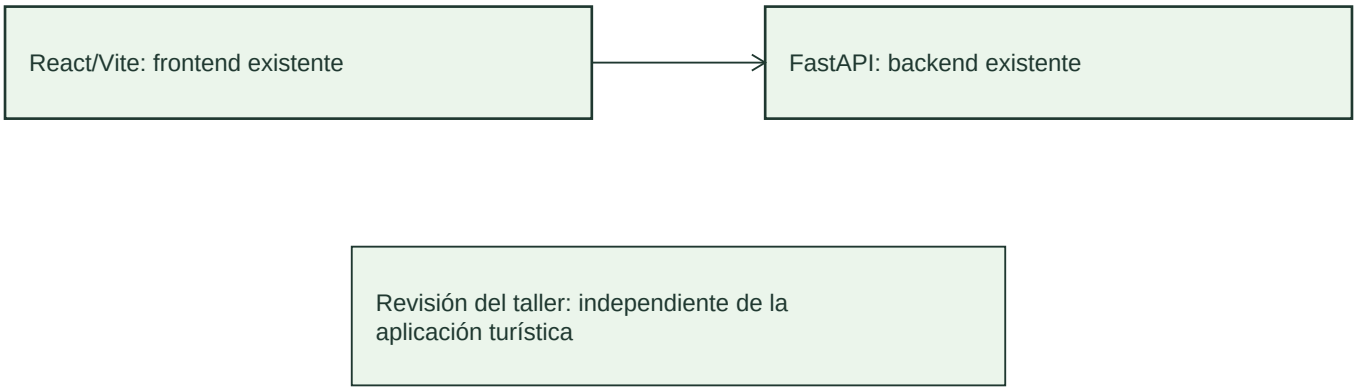
OPITA 360 | Taller 5: Sistema de Agentes

1. Proyecto y herramienta

OPITA 360 permite explorar destinos turísticos del Huila. Su frontend usa React, Vite y lucide-react; el backend está desarrollado en Python con FastAPI y Pydantic. Por ahora, los datos se manejan en memoria. Existe un esquema para PostgreSQL, pero aún no hay conexión persistente.

Para este taller añadimos un sistema de revisión de código con herramientas en Python, Ollama local y un servidor MCP externo. Se mantienen las funciones turísticas, la API, CORS y el esquema SQL. Los cambios siguen sin commit ni push.

El trabajo parte de una especificación, un plan y cuatro tareas. Cada tarea tiene permisos definidos y un contrato que relaciona el reporte con el agente y sus archivos. El plan se elaboró con Codex; no corresponde a un documento generado por Copilot.



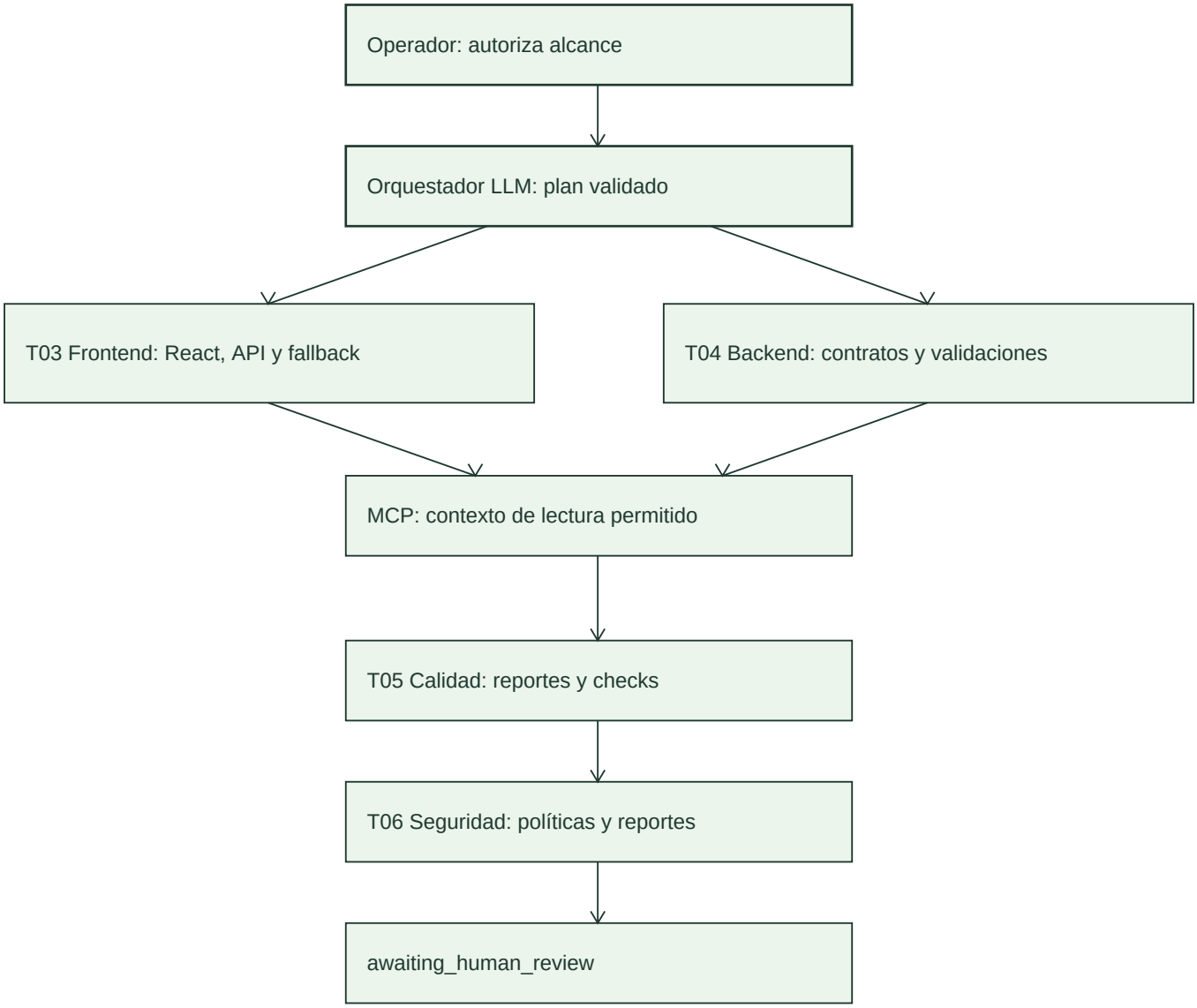
El entorno requiere Python 3.10+ y Ollama en 127.0.0.1:11434, con el modelo qwen3.5:4b. La conexión MCP usa stdio. El servidor externo se ejecuta con Node portátil 24.13.0, sin reemplazar la instalación del sistema. Los modelos no pueden ejecutar comandos libres ni editar la aplicación turística.

La rúbrica asigna 25% a calidad del código, 20% a agentes, 20% a diagramas, 15% a herramientas, permisos y modelo, 10% a diferencias con la referencia y 10% al documento. Esta entrega tiene cuatro páginas y respeta el máximo de ocho.

OPITA 360 | Taller 5: Sistema de Agentes

2. Sistema de agentes

El orquestador y los cuatro especialistas usan qwen3.5:4b en contextos separados. Las trazas registran las llamadas reales al modelo. También se conserva un modo determinista que solo inventaría archivos y no se presenta como una ejecución LLM.



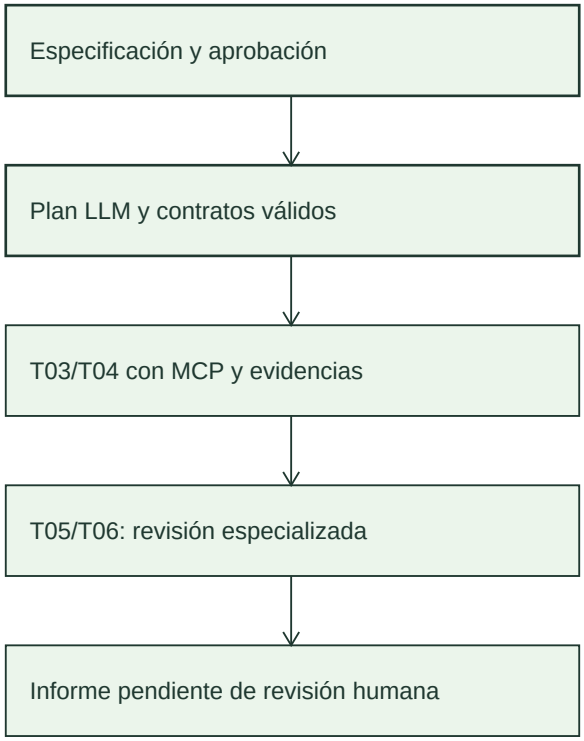
Frontend y backend analizan código obtenido por MCP. Calidad revisa sus reportes y las pruebas ejecutadas por el operador; seguridad revisa las políticas y los resultados anteriores. Las recomendaciones requieren revisión antes de aplicarse.

MCP usa @modelcontextprotocol/server-filesystem@2.3.0 por stdio. Lee una copia autorizada, compara su contenido y comprueba el rechazo de archivos fuera del alcance. El cliente no permite escrituras. La copia temporal no aísla el proceso del sistema operativo.

OPITA 360 | Taller 5: Sistema de Agentes

3. Calidad del código

Las pruebas comprueban la relación entre tarea y agente, la presencia de evidencias, los duplicados, los hashes, los permisos y los límites. Se mantienen los contratos y las funciones existentes. Las respuestas del modelo se contrastan con el código y los resultados de ejecución.



Pasaron 27 pruebas de agentes y 8 de API (35 en total), además de las compilaciones. En Chrome pasaron 11 de 14 comprobaciones antes y las 14 después de corregir la etiqueta de búsqueda, el nombre del selector de días y el estado ARIA del menú. Se probaron filtros, API, modo demo y teclado; browser/ conserva los registros y PNG.

Chrome permitió revisar nombres accesibles, etiquetas, teclado y ancho móvil; falta una auditoría WCAG y un lector de pantalla. Las cinco inferencias LLM y dos sesiones MCP son anteriores a estas correcciones: sus pruebas están en checks-llm-historical.json; checks.json es actual. No se repitieron LLM/MCP ni npm ci en esa actualización.

El LLM se equivocó al cuestionar la validación del presupuesto y negar las pruebas MCP. T05/T06 no recibieron su registro explícito; el asistente contrastó esas afirmaciones con las evidencias del entorno. revision-hallazgos.md explica las discrepancias. La aprobación humana final sigue pendiente; no se autoriza commit ni push.

OPITA 360 | Taller 5: Sistema de Agentes

4. Diferencias con el sistema de referencia

La comparación queda pendiente porque aún no está disponible el sistema de referencia. Se autorizó continuar con el resto del taller, sin atribuir semejanzas o diferencias a un material que no se ha revisado. Por eso no se afirma cumplimiento del 10% de este criterio.

Cuando se reciba la referencia, se compararán arquitectura, roles, modelos, permisos, MCP, aprobación humana y calidad. Cada diferencia se apoyará en archivos, decisiones o ejecuciones de ambos sistemas.

Sí podemos describir los cambios frente a nuestra primera versión: antes solo se inventariaban archivos y los perfiles de modelo estaban vacíos. Ahora hay un modelo local asignado, análisis por especialidad, código obtenido por MCP para frontend y backend, revisión de reportes y trazabilidad de las ejecuciones.

Esta evolución interna no reemplaza la comparación solicitada. Los registros de ejecución permiten comprobar qué se implementó, qué se probó y qué sigue pendiente, sin atribuir características a la referencia desconocida.

La documentación de apoyo está en rubrica.md, especificacion.md, tareas.md, arquitectura.md, entrega.md y resultados.md, junto con las evidencias de pruebas y LLM. Las funciones turísticas se mantienen y PostgreSQL sigue sin conexión.

La aceptación final requiere que una persona revise los cambios, las respuestas del modelo y su relación con las pruebas. Los resultados registrados no sustituyen esa decisión ni permiten dar por ejecutadas las verificaciones pendientes.